

NIDS SENTINEL

Network Intrusion Detection System

Complete Technical Documentation

Generated: 2026-05-29 22:29:40

TABLE OF CONTENTS

- 1. Project Overview & Architecture
- 2. Frontend Features & Components
- 3. Backend API Endpoints
- 4. Database Schema
- 5. Authentication System
- 6. Real-Time Data Collection
- 7. ML Model & Threat Detection
- 8. Light Mode Implementation
- 9. Setup & Installation
- 10. Deployment & Production

1. PROJECT OVERVIEW & ARCHITECTURE

NIDS Sentinel is an AI-powered, real-time network intrusion detection system. It monitors network traffic, detects threats using XGBoost machine learning models, provides real-time analytics, and blocks malicious connections.

Technology Stack:

Component	Technology	Purpose
Frontend	Next.js 16 + React 18	Interactive dashboards
Backend	FastAPI + Python	API & threat processing
Database	SQLite	Data storage
ML Model	XGBoost	Threat classification
Real-time	WebSocket + Polling	Live updates
Visualization	Recharts + Framer Motion	Charts & animations

System Architecture:

Frontend (Next.js Port 3001) → Backend (FastAPI Port 8001) → Database (SQLite) → ML Model ↓ Real-time WebSocket/Polling ← Authentication (JWT) ← Threat Detection & Blocking

2. FRONTEND FEATURES & COMPONENTS

Dashboard: Real-time threat metrics, active connections, detection rates, live traffic visualization

Threat Alerts: Alert management with filtering, blocking capabilities, severity levels

Analytics: Time-filtered threat analysis with charts and distributions

Live Traffic Map: GeoIP-based visualization of attack sources and destinations

ML Analytics: Model performance, accuracy, feature importance, training history

Performance Monitor: System health gauges, CPU/memory usage, detection latency

Recommendations: AI-powered security recommendations based on threats

Settings: Model configuration, thresholds, notification preferences

Light Mode: Dark/Light theme toggle for accessibility

User Authentication: Login, signup, password reset with security questions

Key Frontend Files:

- src/app/login/page.tsx - Login with demo button
- src/app/dashboard/page.tsx - Main dashboard
- src/app/alerts/page.tsx - Alert management
- src/app/analytics/page.tsx - Threat analytics
- src/app/ml/page.tsx - ML metrics
- src/lib/auth.ts - Token management
- src/lib/api.ts - API configuration

3. BACKEND API ENDPOINTS

Method	Endpoint	Auth	Purpose
POST	/api/v1/login/access-token	No	JWT token generation
POST	/api/v1/signup	No	User registration
POST	/api/v1/traffic/log	No	Log network traffic
GET	/api/v1/alerts/stats	Yes	Threat statistics
GET	/api/v1/alerts/timeline	Yes	Threat timeline
GET	/api/v1/alerts/recent	Yes	Recent alerts
POST	/api/v1/actions/block-threat	Yes	Block threat
POST	/api/v1/actions/block-all	Yes	Block all threats
GET	/api/v1/analytics/*	Yes	Threat analysis
GET	/api/v1/models/metrics	Yes	Model performance
GET	/api/v1/recommendations	Yes	Security recommendations

Backend Architecture:

• Database Layer: SQLAlchemy ORM with SQLite • Security: JWT + bcrypt password hashing • Business Logic: Threat detection, blocking, analytics • Real-time: WebSocket connections • Rate Limiting: SlowAPI protection • CORS: localhost:3001 allowed

4. DATABASE SCHEMA & MODELS

Users Table:

Column	Type	Description
id	String(36)	UUID primary key
username	String	Unique username
email	String	User email
hashed_password	String	bcrypt password
is_active	Boolean	Account status
is_superuser	Boolean	Admin flag

Threat_Log Table (6400+ records):

Column	Type	Description
id	String(36)	Threat UUID
src_ip	String	Source IP
dst_ip	String	Destination IP
protocol	String	TCP/UDP
label	String	DoS/DDoS/Probe/R2L/U2R/Normal
severity_score	Float	ML confidence (0-10)
is_blocked	Boolean	Block status
timestamp	DateTime	Detection time
geoip_src	String	Source location

5. AUTHENTICATION SYSTEM

Authentication Flow:

1. User enters credentials on login page 2. POST to /api/v1/login/access-token 3. Backend verifies against bcrypt hashed password 4. Returns JWT token (exp: 24 hours) 5. Token stored in localStorage 6. Sent in Authorization header for protected requests 7. Backend verifies JWT signature and user exists

Security Features:

- bcrypt password hashing (10 rounds)
- JWT tokens with HS256 algorithm
- CORS protection (localhost:3001 only)
- Rate limiting on login attempts
- Password reset with security questions
- Active user status checking

Demo Login (NEW FEATURE): The login form includes a "Demo Login" button that:

- Auto-fills: username=admin, password=admin123
- Submits directly to backend
- Returns valid JWT token
- Stores token in localStorage
- Redirects to dashboard
- No form integration issues

- works reliably

6. REAL-TIME DATA COLLECTION

How Data Collection Works:

FRONTEND POLLING: • Dashboard polls `/api/v1/alerts/stats` every 5 seconds • Alerts page polls recent threats every 15 seconds • Analytics polls distribution data every 30 seconds • WebSocket connection for live threat streaming (optional) BACKEND PROCESSING: • Threat logs written continuously to database • ML model classifies threats in real-time • Severity scores from XGBoost confidence • GeoIP enrichment for location data DATA PERSISTENCE: • Each threat logged to `threat_log` table (6400+ existing) • Timestamp recorded for timeline analysis • Severity scores stored for ML training • Block status tracked for effectiveness metrics CONTINUOUS OPERATION: Even after stopping dataset input: • Database stores all historical threat data • Backend APIs serve data from persistent storage • ML model continues running for new threats • Real-time collection depends on data source • Without external input, system monitors at rest • Can import new data via `/api/v1/traffic/log` DATA FLOW: Network Traffic → Threat Detection (ML) → Database Storage → Real-time APIs → Frontend Dashboard Display

7. ML MODEL & THREAT DETECTION

XGBoost Model Details:

MODEL ARCHITECTURE: • Framework: XGBoost Gradient Boosting • Training Data: NSL-KDD dataset (148,517 samples) • Features: 41 network traffic characteristics • Classes: 6 threat types + normal traffic • Accuracy: 99.82% • Real-time Performance: <50ms per packet THREAT CLASSIFICATIONS: • Normal: Baseline traffic (no action needed) • DoS: Denial of Service (block immediately) • DDoS: Distributed DoS (block all flows) • Probe: Reconnaissance/scanning (monitor) • R2L: Remote to Local (alert + log) • U2R: User to Root privilege escalation (block) SEVERITY SCORING: • Normal: 0-2 (no action) • Probe: 3-4 (monitor & log) • R2L/U2R: 6-8 (alert & block) • DDoS/DoS: 8-10 (immediate block) TOP FEATURES: 1. srv_error_rate (18%) - Error rate indicates attack 2. dst_bytes (15%) - Data volume reveals DDoS 3. src_bytes (12%) - Traffic pattern analysis 4. count (11%) - Connection frequency 5. duration (9%) - Session length timing MODEL PERFORMANCE METRICS: • Precision: 99.7% (few false positives) • Recall: 99.5% (catches most threats) • F1-Score: 99.6% (balanced performance) • False Positive Rate: 0.3%

8. LIGHT MODE IMPLEMENTATION

How Light Mode Works:

```
TECHNOLOGY: • Tailwind CSS dark mode support • CSS custom properties for theming • Next.js  
prefers-color-scheme detection • localStorage persistence IMPLEMENTATION: 1. Toggle button in navigation  
(sun/moon icon) 2. Stores preference in localStorage 3. Loads on page refresh 4. Applies 'dark' class to  
document root 5. Tailwind applies dark: prefixed styles COLOR SCHEMES: DARK MODE (Default): • Background:  
#0f172a (dark slate) • Cards: rgba(15, 23, 42, 0.8) with blur • Text: white/light gray • Accents: Purple,  
Pink, Cyan gradients LIGHT MODE: • Background: #ffffff (white) • Cards: #f8fafc (light slate) • Text:  
#1e293b (dark slate) • Same accent colors AFFECTED COMPONENTS: • Dashboard cards and metrics • Charts and  
graph colors • Navigation sidebar • Form inputs and buttons • Modal dialogs • Table backgrounds and text •  
All interactive elements • Automatic contrast adjustment CODE EXAMPLE: // Toggle theme const toggleTheme =  
() => { const isDark = document.documentElement.classList.toggle('dark'); localStorage.setItem('theme',  
isDark ? 'dark' : 'light'); } // Tailwind CSS: Text
```

9. SETUP & INSTALLATION

Prerequisites:

• Node.js 18+ and npm • Python 3.9+ • SQLite3 • Git

Backend Setup:

```
cd backend python -m venv venv source venv/bin/activate pip install -r requirements.txt python seed_db.py  
python run_backend.py # Runs on http://localhost:8001
```

Frontend Setup:

```
cd frontend npm install echo "NEXT_PUBLIC_API_URL=http://localhost:8001" > .env.local npm run dev --  
--port 3001 # Opens at http://localhost:3001
```

Access Dashboard:

1. Go to <http://localhost:3001/login> 2. Click "Demo Login" button 3. Dashboard loads with threat data

10. DEPLOYMENT & PRODUCTION

Production Build - Frontend:

```
cd frontend npm run build npm start # Or deploy to Vercel: npm i -g vercel vercel
```

Production Build - Backend:

```
pip freeze > requirements-prod.txt pip install gunicorn gunicorn -w 4 -b 0.0.0.0:8001 app.main:app # Or  
with Docker: docker build -t nids-backend . docker run -p 8001:8001 nids-backend
```

Environment Variables:

```
BACKEND: DATABASE_URL=sqlite:///./nids.db SECRET_KEY=your-secret-key CORS_ORIGINS=http://localhost:3001  
FRONTEND: NEXT_PUBLIC_API_URL=http://localhost:8001
```

NIDS Sentinel v1.0

Generated: 2026-05-29 22:29:40